FRANCESCCA ROSSI - 27 OTTOBRE 2026

TRACING, SPIEGATO BENE

Dentro la *trace*.

Come Langfuse registra, struttura e rende leggibile tutto ciò che accade in una richiesta LLM — dal prompt alla risposta.

Un'app LLM non è una funzione.

Stesso input, risposte diverse. Tra prompt, retrieval, tool e modello, il percorso è il prodotto — e senza traccia resta invisibile.

QUELLO CHE VEDI

POST /chat — 200 OK · 1,8 s

USER

Dove è il mio ordine?

ASSISTANT

In consegna domani, 10:00–12:00.

Input e output. Il resto è invisibile.

QUELLO CHE TI SERVE

01 **Quale prompt è stato inviato davvero?**
System, contesto, versione del template.

02 **Quali documenti ha recuperato il retriever?**
E con quale similarità rispetto alla domanda.

03 **Quante chiamate al modello, in che ordine?**
Ogni iterazione dell'agente, non solo l'ultima.

04 **Quanto è costata e dove ha perso tempo?**
Token, latenza per passo, collo di bottiglia.

↳ Il tracing non registra solo il risultato: ricostruisce la catena di causa ed effetto.

Una richiesta, tutti i suoi passaggi.

Una trace è l'unità di lavoro completa; le observation sono i singoli passi, annidati come in un albero.

TRACE · chat-turn 2,4 s · $0,004

TRACE	chat-turn	2,4 s
└ SPAN	load-history	60 ms
└ RETRIEVER	retrieve-context	310 ms
└ EMBEDDING	embed-query	120 ms
└ GENERATION	generate-answer	1,6 s · 1.596 tok
└ SPAN	post-process	40 ms

TRACE

La richiesta intera.

Tutte le observation con lo stesso `trace_id`.

OBSERVATION

Un passo.

Chiamata LLM, tool, retrieval, evento. Può contenerne altri.

SESSION

La conversazione.

Più trace della stessa chat, unite dal `session_id`.

Tre livelli, una sola gerarchia.

Le sessioni raggruppano le trace, le trace raggruppano le observation. Gli attributi di trace vengono copiati su ogni observation.

SESSION · chat-42 2 turni · utente u_482

TRACE · turn-1 2,4 s

- retrieve-context RETRIEVER
- generate-answer GENERATION
- post-process SPAN

TRACE · turn-2 1,9 s

- load-history SPAN
- generate-answer GENERATION
- search-orders TOOL

ATTRIBUTI DI TRACE, PROPAGATI A OGNI OBSERVATION

`user_id` `session_id` `tags` `metadata` `environment` `release`

↳ Le observation sono la tabella che interroghi; gli attributi di trace viaggiano con loro.

Cosa c'è dentro, passo per passo.

Un turno di chat con RAG. Ogni riga è un'observation; il tipo decide quali dati porta con sé.

TRACE · chat-turn — 2,4 s · 5 observation

TRACE	chat-turn	2,4 s
└ EMBEDDING	embed-query	120 ms
└ RETRIEVER	retrieve-context	310 ms · 5 doc
└ GENERATION	generate-answer	1,6 s · gpt-4o
└ SPAN	validate-answer	80 ms

DETTAGLIO OBSERVATION

generate-answer · generation

MODELLO
gpt-4o

TOKEN
1.284 → 312

COSTO
$0,0042

LATENZA
1,6 s

INPUT

Dove è il mio ordine?

5 documenti allegati al contesto

OUTPUT

In consegna domani, 10:00–12:00.

↳ Le generation portano modello, token e costo; ogni altro tipo porta ciò che serve al suo mestiere.

La traccia viaggia in background.

Il codice crea observation; l'SDK le accoda e le invia in batch. La risposta all'utente non aspetta la telemetria.

01

Strumenta

SDK Langfuse, integrazione del framework o OTLP.

→

02

Registra

Ogni passo diventa un'observation con input, output e durata.

→

03

Accoda

Gli eventi restano in un buffer locale, fuori dalla richiesta.

→

04

Esporta

Un exporter svuota il buffer verso l'API Langfuse.

PROCESSI LONG-RUNNING

L'exporter svuota il buffer da solo: su timer o a soglia.

PROCESSI SHORT-LIVED

Chiama `flush()` prima di uscire, o la coda va persa.

↳ Sotto il cofano c'è OpenTelemetry: le stesse trace possono andare anche ad altri backend.

Il tipo giusto accende le funzioni giuste.

Il tipo non è un'etichetta: decide quali dati Langfuse sa leggere e quali analisi può fare.

GENERATION

La chiamata al modello.

Prompt, output, token e costo per ogni invocazione.

→ costi e confronti tra modelli

SPAN

Un'unità di lavoro.

Durata e gerarchia: il contenitore degli altri passi.

→ latenza e collo di bottiglia

EVENT

Un fatto puntuale.

Cache hit, retry, stato: segnali senza durata.

→ retry, cache e stati

TOOL

Un'azione esterna.

Funzioni e API: cosa ha chiesto l'agente, e cosa è tornato.

→ filtri e valutazioni sui tool

RETRIEVER

Una ricerca.

Vector store o database: query e documenti recuperati.

→ qualità del contesto

EMBEDDING

Un vettore.

Chiamata di embedding, con modello, token e costo.

→ costi di indicizzazione

↳ Altri tipi disponibili: agent, chain, evaluator, guardrail.

La cache: non ricalcolare ciò che sai già.

Una cache è una copia temporanea di un risultato già prodotto. Se la richiesta è la stessa — o abbastanza simile — la riusiamo invece di rifare il lavoro.

SENZA CACHE

Richiesta → Modello → Risposta

1,8 s · $0,004

Ogni richiesta paga token, latenza e calcolo.

CON CACHE · HIT

Richiesta → Cache → Risposta

40 ms · $0,000

Hit: risposta pronta. Miss: calcoli e poi salvi.

CACHE ESATTA

Stessa chiave, stessa risposta.

La chiave è l'hash di prompt, modello e parametri. Una virgola diversa è un miss.

`hash(prompt + params)`

CACHE SEMANTICA

Query simili, stessa risposta.

Si cercano prompt già visti con gli embedding. Sopra soglia si riusa: va misurata.

`similarità > 0,95`

CACHE DEL PROVIDER

Il modello riusa il prefisso.

Prompt caching: system prompt e contesto lungo vengono rielaborati una volta sola.

`cached_tokens`

COME SI VEDE IN LANGFUSE

`event` — cache-hit o cache-miss, con chiave e TTL.

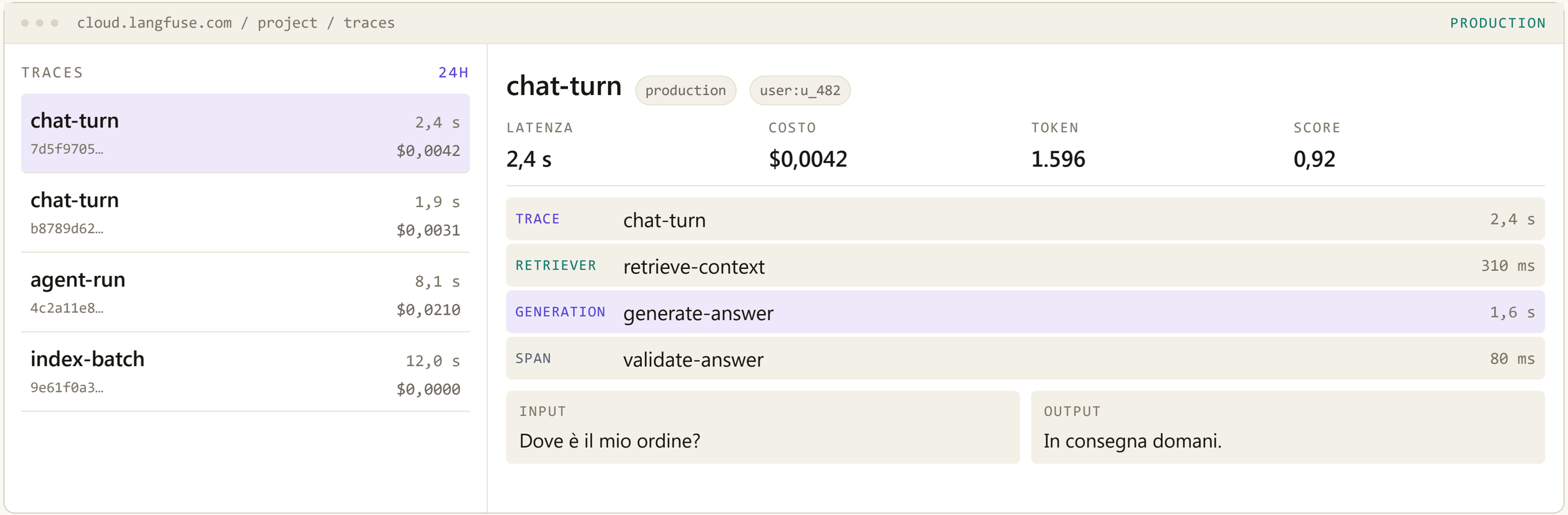
`generation` — token cachati: il costo di input scende.

`hit vs miss` — confronta latenza e costo per nome.

↳ Chiave, TTL (per quanto tempo vale) e invalidazione: senza queste tre cose, la cache è un bug che aspetta.

Dalla trace alla risposta.

La UI è lo stesso albero, reso navigabile: filtra, apri un passo, leggi input e output senza perdere il contesto.



Una trace buona si progetta.

Cinque regole che tengono in piedi debug, valutazioni e dashboard anche quando l'app cambia.

01	**Una trace, un'unità di lavoro.**	Un turno di chat, un run dell'agente, una pipeline. Non un'intera sessione.	`chat-turn`
02	**Nomi stabili, verbo prima.**	Bassa cardinalità: niente ID o retry nel nome. I nomi sono un contratto.	`retrieve-context`
03	**Una generation per chiamata.**	Non aggregare il loop dell'agente: perdi decisioni, token e costi.	`generate-answer`
04	**Input e output leggibili.**	La root observation è ciò che vedi in tabella e valutano gli evaluator.	`user → assistant`
05	**Attributi giusti, da subito.**	user_id, session_id, tags, environment: tagliano ogni vista e dashboard.	`env=production`

La prima trace è a tre passi.

Cloud o self-hosted: la strumentazione non cambia. Bastano le chiavi giuste e una chiamata.

01 **Crea progetto e chiavi.**

Pubblica e segreta, in variabili d'ambiente.

02 **Strumenta una chiamata.**

SDK Langfuse, integrazione del framework o OTLP.

03 **Apri la trace.**

Albero, tempi, token, costo e output in un colpo d'occhio.

python — first_trace.py

```python
from langfuse import get_client

langfuse = get_client()

with langfuse.start_as_current_observation(
    as_type="span", name="chat-turn"
) as span:
    span.update(input=question)
    with langfuse.start_as_current_observation(
        as_type="generation",
        name="generate-answer",
        model="gpt-4o",
    ) as gen:
        gen.update(output=answer)

langfuse.flush()
```

→ langfuse.com/docs/observability/get-started

Grazie.